

RESEARCH PROPOSAL

Submitted By	Haider Niaz (22i-0481), Abdullah Mazhar (22i-0622), Hania Shafqat (22i-2148)
Domain	Machine Unlearning Privacy & Security in AI

1. Selected Research Paper

Title	Fast Yet Effective Machine Unlearning
Authors	Ayush K Tarun, Vikram S Chundawat, Murari Mandal, Mohan Kankanhalli
Source	arXiv:2111.08947v5 [cs.LG], May 2023

2. Problem Statement & Motivation

Modern machine learning models are trained on vast amounts of personal data. Privacy regulations such as GDPR and CCPA grant individuals the legal right to request deletion of their data from deployed systems. Naively retraining a model from scratch after every such request is computationally prohibitive — a problem that becomes increasingly acute as models and datasets grow larger.

The paper addresses two core questions: (i) Can class-level data be unlearned from a trained deep network without ever revisiting the original forget-set samples? (ii) Can this process be made fast enough to be practical at scale, and generalised across diverse deep-learning architectures?

Existing unlearning methods either target convex models, impose prohibitive computational costs (e.g., Hessian approximation), or fail to support multi-class forgetting. There is a clear gap for an efficient, zero-glance, architecture-agnostic unlearning solution for deep neural networks — the precise gap this paper fills with its UNSIR framework.

3. Overview of the Methodology

The proposed framework — UNSIR (Unlearning by Selective Impair and Repair) — operates in three sequential phases:

Phase 1 — Error-Maximizing Noise Generation

For each class to be forgotten, a noise matrix $\mathbf{N}$ (same size as the model input) is optimized by maximizing the model’s classification loss for that class while keeping the model weights frozen. The objective minimizes:

$$\arg \min_{\theta} E(\theta)[-L(f, y) + \lambda \|\mathbf{w}_{noise}\|]$$

where $L(f, y)$ is the cross-entropy loss, f is the pretrained model, and λ controls regularization. This regularization prevents the noise from growing arbitrarily large, ensuring it captures meaningful “anti-patterns” instead of extreme pixel values.

The noise matrix is initialized from a normal distribution and optimized for ~40 steps using the model’s loss signal, with only $\mathbf{N}$ being updated. For multi-class unlearning, a separate noise matrix is learned per class. This process is very fast

(~1.1 seconds on ResNet18 + CIFAR-10) and produces noise that acts as an **anti-sample**, effectively suppressing the model's learned representation of the forget class.

Phase 2 — Impair Step

The impair step fine-tunes the model for one epoch on the union of a small retain subset (D_{r_sub}) and the noise matrix N , i.e., $D_{r_sub} \cup N$. A high learning rate (0.02 for CIFAR-10) is used to aggressively update weights, forcing rapid suppression of the forget-class representation. Since N encodes an anti-pattern of the forget class, training on it pushes the model away from the original decision boundary, while D_{r_sub} helps stabilize overall representations and prevents full collapse.

After this single epoch, forget-class accuracy drops to ~0%, while retain accuracy decreases slightly and is later restored in the repair step. The impair step takes ~1.70 seconds on ResNet18 with CIFAR-10. For multi-class unlearning, all noise matrices are merged into a single pass, keeping compute cost constant regardless of the number of forget classes, which is a key scalability benefit of UNSIR.

Phase 3 — Repair Step

In the **repair step**, the impaired model is fine-tuned for one epoch using only the retain subset (D_{r_sub}), with a lower learning rate (e.g., 0.01). This step restores performance on the remaining classes without reintroducing any information about the forgotten class, since no forget-set samples are used.

While the impair step disrupts weights linked to the target class, it can also affect shared features. Training on (D_{r_sub}) helps recover these features while preserving the forgetting effect. Usually, one repair epoch is sufficient, though additional cycles can be applied if needed. The full impair–repair process is extremely fast (under ~4 seconds total), significantly outperforming retraining or other methods, while maintaining key advantages like zero-glance privacy, model flexibility, and efficient multi-class unlearning.

4. Implementation Plan

Tools & Libraries

- PyTorch — model training, gradient computation, weight manipulation
- torchvision — dataset loading (CIFAR-10/100), pretrained model weights
- pytorch-grad-cam — GradCAM heatmap visualisation of unlearning effect
- NumPy / Matplotlib / Seaborn — numerical analysis, result plotting, distribution plots
- Google Colab / local GPU — execution environment

Datasets

- CIFAR-10 — primary benchmark; 10-class image classification (60,000 images, 32×32)
- CIFAR-100 — large-scale multi-class unlearning evaluation (100 classes)
- VGGFace-100 subset — face recognition unlearning (if compute permits)

Implementation Steps

- **Weeks 1–2: Reproduce Baseline**

- Train ResNet18 on CIFAR-10 for ~40 epochs (SGD with momentum), targeting ~78% test accuracy, and prepare the retain set (D_r) and forget set (D_f) as per the paper (e.g., class 0 for unlearning).
 - Reproduce baseline unlearning methods (Fine-Tune and NegGrad), recording key metrics (ADf , ADr , RT), while building a modular codebase for training, unlearning, and evaluation to support future extensions.
- **Week 2: Noise Generation Module**
 - Implement the error-maximizing noise optimization (cross-entropy + L2 regularization, $\lambda = 0.1$), initializing $N \sim N(0,1)$ and updating it for ~40 steps with frozen model weights.
 - Validate convergence by tracking loss and inspecting the generated noise and extend the module to support **multiple forget classes** by learning separate noise matrices for each.
- **Week 3: UNSIR Impair-Repair Pipeline**
 - Implement the impair and repair loops: train for 1 epoch on ($D_{r_sub} + N$) with a high learning rate (0.02), followed by 1 epoch on (D_{r_sub}) only with a lower learning rate (0.01), tracking ADf and ADr after each step.
 - Validate results against the paper (target $ADf \approx 0\%$, ADr close to original) and extend to support multi-cycle runs while profiling runtime to ensure the full pipeline remains under ~4 seconds.
- **Week 4: Multi-Class & Large-Scale Extension**
 - Extend the pipeline to **multi-class unlearning on CIFAR-10** (e.g., 2, 4, 7 classes in a single pass), and scale to **CIFAR-100 with pretrained ResNet18** to unlearn multiple classes (e.g., 1, 20, 40).
 - Evaluate across architectures (e.g., MobileNetV2, optionally ViT if feasible) to test generalization, and document key training settings such as layer-wise learning rates for pretrained models.
- **Week 5: Evaluation, Ablations & Visualisation**
 - Compute metrics: ADf , ADr , RT , and layer-wise weight distance across all experiments.
 - Run ablations: learning rate (0.001–1), λ values (0–10), and retain-data proportions (1%–60%).
 - Generate visualizations: GradCAM heatmaps, prediction distributions, and runtime comparison charts.

5. Expected Proof of Concept (POC)

The deliverable POC will demonstrate the following, presented as a reproducible Jupyter notebook:

- ResNet18 trained on CIFAR-10 with ~78% accuracy.
- Single-class unlearning where forget accuracy drops to ~0% and retain accuracy remains >70%.
- Relearn time (RT) of 90+ epochs confirming strong erasure.
- GradCAM heatmaps showing the model lose focus on the forget class after UNSIR.
- (Stretch goal) multi-class unlearning on CIFAR-100 with 20 and 40 forget classes.

Success will be measured against the quantitative thresholds reported in the paper: $ADf \approx 0\%$, ADr within 5–8% of the original model, and RT substantially higher than baseline unlearning methods.
